

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
**Ордена Трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования**
«Московский технический университет связи и информатики»

Кафедра Программная инженерия

ОТЧЕТ
по учебной практике(технологической)

Выполнил студент группы БПИ2403

Пикуза Дмитрий Алексеевич

Проверила: Мосева М.С.

Москва
2026

ВВЕДЕНИЕ

В период с 23.06.2026 по 06.07.2026 была пройдена учебная практика по разработке программного обеспечения. В рамках практики выполнялась роль инженера по эксплуатации и автоматизации (DevOps Engineer) в командном проекте интеллектуальной поисковой системы по внутренней базе знаний университета. Целью практики было получение практических навыков контейнеризации микросервисного веб-приложения, проектирования и настройки конвейера непрерывной интеграции и доставки (CI/CD), организации системы мониторинга и управления окружением, а также обеспечение воспроизводимого развёртывания всего программного стека одной командой.

ЭТАП 1: Проектирование инфраструктуры и оркестрация базовых сервисов (23.06.2026 – 24.06.2026)

Период: 2 дня

***Задание:** Изучение технической документации проекта, проектирование схемы развёртывания и написание базового файла оркестрации для системы хранения данных и поискового движка.*

Содержание работы: Были изучены цели и задачи проекта, микросервисная архитектура приложения и распределение обязанностей в команде. На основе требований раздела DevOps (DO-01 – DO-07) спроектирована схема развёртывания. Написан базовый файл docker-compose.yml для запуска инфраструктурных сервисов — PostgreSQL (postgres) и Elasticsearch (elasticsearch) в изолированной сети. Elasticsearch сконфигурирован в режиме single-node с отключённой защитой для локальной среды и ограничением памяти JVM. Для всех сервисов заданы именованные тома (volumes) для сохранения данных между перезапусками. Все конфиденциальные данные (учётные записи БД, пароли) вынесены в переменные окружения;

подготовлены файлы .env и .env.example, последний добавлен в репозиторий (DO-03, DO-04).

ЭТАП 2: Контейнеризация бэкенд- и фронтенд-приложений (25.06.2026 – 26.06.2026)

Период: 2 дня

Задание: *Разработка образов для сборки бэкенд- и фронтенд-приложений и их интеграция в общий файл оркестрации.*

Содержание работы: Разработан Dockerfile для сборки образа бэкенда (Python + FastAPI): выбран компактный базовый образ, настроена установка зависимостей из requirements.txt с кешированием слоёв и запуск приложения. Написан многоступенчатый (multi-stage) Dockerfile для фронтенда (Node.js + сборка статики с последующей отдачей через Nginx), что позволило существенно уменьшить итоговый размер образа. Сервисы бэкенда (app) и фронтенда (front) добавлены в docker-compose.yml с пробросом портов и передачей адресов зависимых сервисов через переменные окружения (DO-01, DO-02).

ЭТАП 3: Интеграционная сборка полного стека через Docker Compose (27.06.2026)

Период: 1 день

Задание: *Настройка порядка запуска, проверок работоспособности сервисов и запуск полного программного стека одной командой.*

Содержание работы: Осуществлён комплексный запуск всех микросервисов поисковой системы (app, front, postgres, elasticsearch, redis) в единой изолированной сети командой docker compose up --build. Для контроля готовности сервисов настроены проверки работоспособности (healthcheck) и управляемый порядок старта через директиву depends_on с условием

service_healthy — бэкенд стартует только после того, как база данных, Elasticsearch и Redis перешли в рабочее состояние. Для всех сервисов задана политика перезапуска restart: unless-stopped. В ходе первой интеграции выявлены и устранены ошибки конфигурации сети и CORS-политик, блокировавшие клиент-серверное взаимодействие на раннем этапе сборки (DO-03).

ЭТАП 4: Настройка конвейера непрерывной интеграции и доставки CI/CD (29.06.2026 – 30.06.2026)

Период: 2 дня

***Задание:** Проектирование и настройка пайплайна GitHub Actions для автоматического запуска линтеров, тестов и сборки образов.*

Содержание работы: Спроектирован и реализован конвейер CI/CD на базе GitHub Actions (файл .github/workflows/ci.yml), запускающийся при пуше в ветку main и при создании пул-реквестов. Настроены параллельные задания: для бэкенда — установка Python 3.12 с кешированием зависимостей, проверка линтером ruff и прогон тестов pytest со сбором покрытия кода; для фронтенда — установка Node.js, проверка eslint и сборка проекта. Добавлено завершающее задание сборки Docker-образов бэкенда и фронтенда, которое выполняется только при успешном прохождении всех проверок и только на ветке main. Тем самым обеспечено автоматическое поддержание качества кода и воспроизводимость сборки (DO-05).

ЭТАП 5: Настройка кеширования и системы мониторинга (01.07.2026 – 02.07.2026)

Период: 2 дня

***Задание:** Подключение кеширующего слоя и развёртывание системы сбора и визуализации метрик приложения.*

Содержание работы: В файл оркестрации добавлен сервис Redis (redis) с собственным томом и проверкой работоспособности для использования бэкендом в качестве кеша частых поисковых запросов. Развёрнута базовая система мониторинга: подключён Prometheus для сбора метрик с эндпоинта /metrics бэкенда (количество запросов к /search, среднее время ответа) с интервалом опроса 15 секунд, и Grafana для визуализации. Настроена автоматическая инициализация (provisioning) источника данных Prometheus и дашборда поисковой системы в Grafana, что позволяет получить готовые графики сразу после запуска стека. Проверена стабильность работы контейнеров и корректность собираемых метрик (DO-06).

ЭТАП 6: Автоматизация наполнения системы и разработка документации (03.07.2026 – 04.07.2026)

Период: 2 дня

***Задание:** Написание скрипта автоматической инициализации системы тестовыми данными и составление технической документации по развёртыванию.*

Содержание работы: Разработан скрипт инициализации init.sh, который автоматически скачивает 10 тестовых PDF-документов (научные статьи из открытого доступа arXiv) и загружает их в систему через REST API (POST /api/v1/documents/upload) с осмысленными именами файлов, пригодными для последующей оценки качества поиска. Скрипт предварительно дожидается готовности бэкенда через эндпоинт /health и корректно обрабатывает ошибки загрузки. Подготовлен и оформлен файл README.md с описанием архитектуры, стека технологий, структуры репозитория и пошаговой инструкцией по развёртыванию проекта одной командой (DO-07).

ЭТАП 7: Финальная проверка развёртывания и подготовка отчётности

(06.07.2026)

Период: 1 день

Задание: *Итоговая проверка воспроизводимости развёртывания, работоспособности CI/CD и подготовка отчётной документации.*

Содержание работы: Выполнена итоговая проверка развёртывания всего программного стека «с нуля» одной командой на чистом окружении с последующим автоматическим наполнением данными через init.sh. Проверена работоспособность конвейера CI/CD в GitHub Actions на финальной версии кода: линтеры, тесты и сборка образов проходят успешно. Проконтролирована корректность передачи всех секретов через переменные окружения и отсутствие конфиденциальных данных в репозитории. Сформирована финальная отчётность по инфраструктурной части проекта и составлен список приобретённых ИТ-компетенций.

ЗАКЛЮЧЕНИЕ

В ходе учебной практики были успешно выполнены все запланированные этапы работы. В рамках роли DevOps Engineer обеспечена полная контейнеризация микросервисной поисковой системы и её воспроизводимое развёртывание одной командой `docker compose up --build`. Настроен конвейер непрерывной интеграции и доставки на базе GitHub Actions с автоматическим запуском линтеров, тестов и сборки образов. Развёрнута система мониторинга на базе Prometheus и Grafana, организовано кеширование через Redis, автоматизировано наполнение системы тестовыми данными. Все конфиденциальные данные вынесены в переменные окружения. Инфраструктура задокументирована и полностью готова к демонстрации и эксплуатации.

Приобретённые навыки: контейнеризация приложений (Docker, multi-stage сборка, Nginx), оркестрация сред через Docker Compose (healthcheck, depends_on, volumes, сети), настройка CI/CD-конвейеров (GitHub Actions, линтеры ruff и eslint, автоматическая сборка образов), организация мониторинга (Prometheus, Grafana), работа с кеширующим слоем (Redis), управление конфигурацией через переменные окружения, автоматизация развёртывания (bash-скрипты, REST API), составление технической документации (README), работа с Git.

Статистика: Общая продолжительность практики — 12 дней, количество этапов — 7, контейнеризовано микросервисов: 7 (app, front, postgres, elasticsearch, redis, prometheus, grafana), разработано Docker-образов: 2 (бэкенд, фронтенд), настроено заданий CI/CD: 3, автоматически загружаемых тестовых документов: 10, подготовлено документов технической документации: 1 (README).

Ссылка на репозиторий: <https://github.com/JeTsEt627/Fieldwork.-2-nd-year-BPI2403>